

Team Members:

Nguyễn Võ Đức Huy (Back-end Logic) - MAMAIU25010

Nguyễn Ngọc Hải (GUI/Swing) - MAMAIU25009

I. Introduction

1. Project Overview

This project presents a simplified digital version of Monopoly developed in Java. The goal was to create a functional, interactive board game that maintains the core strategic elements of the original while streamlining complex mechanics for a smoother software experience.

2. Game rules:

2.1 Core Game Rules:

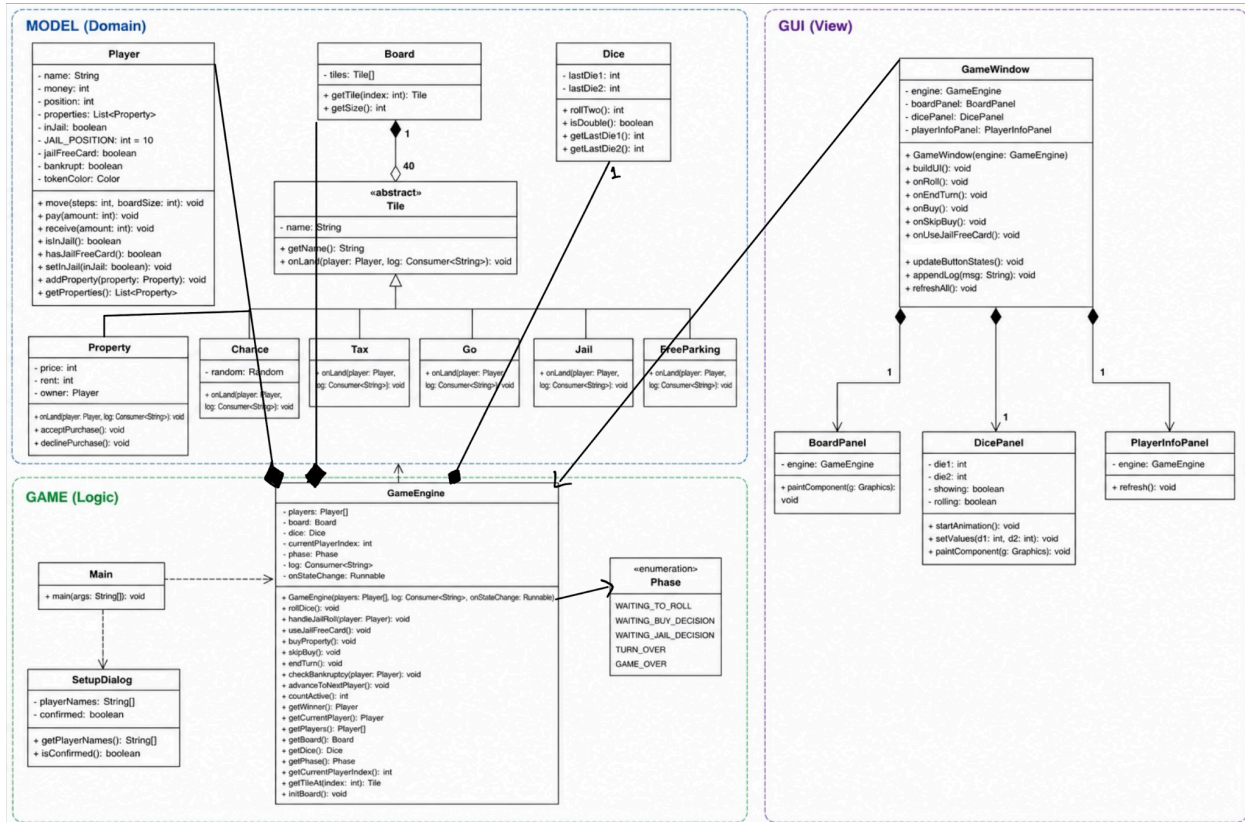
- - Initial Setup: Every player begins the game with a starting balance of \$1500.
- - Movement & GO: Players roll 2 dices to move. Landing on or passing the GO square grants the player \$200.
- - Property Mechanics: Players may purchase unowned properties they land on. However, to simplify gameplay, the ability to buy houses has been removed.
- - Rent: If a player lands on a property owned by another player, they must pay a predetermined rent amount to the owner.
- - Bankruptcy & Winning: A player is eliminated if their balance becomes negative. The last remaining player with a non-negative balance is declared the winner.

2.2 Board Squares & Events:

- - Tax Tiles: The board includes several types of financial penalties: Income Tax, Gift Tax, Luxury Tax, and Supertax.
- - Jail Mechanics: Players can land on the "Visit Jail" square safely, but landing on the Go To Jail square sends the player directly to the Jail position.

- - Free Parking: A neutral square where no actions or financial transactions occur.
- - Chance System: Landing on a Chance square triggers one of 10 randomized events:
 1. Bank Rewards: Receive \$100 (Bank pays you) or \$150 (Bank error).
 2. Fines & Repairs: Pay a flat \$50 fine or property repairs costing \$100 per owned property.
 3. Movement: Advance to GO (Collect \$200), move forward 3 spaces, move back 3 spaces, or teleport to a random property tile for a "Vacation".
 4. Jail Actions: Go directly to Jail or receive a "Get Out of Jail Free" card.

II. UML



III. Implementation

This section details the technical implementation of the Việt Nam Monopoly game, covering the project structure, class-level design decisions, the GUI architecture, the game state machine, and specific implementation choices made to translate the original console-based prototype into a fully interactive Swing application.

1. Project Structure

The project is organised into four packages, each with a well-defined responsibility:

- **game**
Classes: GameEngine, Main
Responsibility: Core game logic, turn management, board initialisation, and application entry point
- **model**
Classes: Player, Board, Dice
Responsibility: Data-holding model classes representing game entities
- **tile**
Classes: Tile (abstract), Property, Go, Jail, FreeParking, GoToJail, Tax, Chance
Responsibility: Polymorphic tile hierarchy; each subclass defines its own landing behaviour
- **gui**
Classes: GameWindow, BoardPanel, DicePanel, PlayerInfoPanel, SetupDialog
Responsibility: All Swing components; entirely separated from game logic

This four-package layout enforces a clean separation between domain logic (game, model, tile) and presentation (gui). The gui package has a one-way dependency on game and model — no game logic class imports anything from gui. This made it straightforward to replace the original Scanner-based I/O with event-driven Swing buttons without modifying any gameplay code.

2. Core Model Classes

2.1 Player:

The Player class stores all per-player state: name, balance, board position, owned properties, jail status, and jail-free card possession. Two design decisions are worth noting:

- **Automatic GO bonus:** The `move(int steps, int boardSize)` method computes the new position using modular arithmetic. If the resulting position is numerically less than the starting position, the \$200 GO bonus is automatically credited.
- **Bankruptcy flag:** The `pay(int amount)` method sets a boolean bankrupt flag when the balance drops below zero. `GameEngine` removes the player at the end of their turn if bankrupt.

Each player is assigned a unique token colour used in both the board and UI.

2.2 Dice

The Dice class wraps a random generator and provides:

- `rollTwo()` — rolls both dice, stores values, returns sum
- `isDouble()` — checks if both dice match

2.3 Board

Board is a wrapper around a fixed 40-tile array. It assigns itself to Chance tiles so they can interact with the board dynamically.

3.Tile Class Hierarchy

All tiles extend an abstract Tile class:

```
public abstract void onLand(Player player, Consumer<String> log);
```

This removes all console printing and uses a logging callback instead.

Tile Types:

- **Go** — gives \$200 if landed on
- **Jail** — no effect
- **Free Parking** — no effect
- **Go To Jail** — sends player to jail
- **Tax** — deducts money
- **Chance** — random event
- **Property** — buy or pay rent

4. Game Logic - Game Engine

GameEngine controls all game state and logic.

4.1 Turn Phase State Machine

Game flow uses a Phase enum:

- WAITING_TO_ROLL → Roll dice
- WAITING_BUY_DECISION → Buy or skip
- WAITING_JAIL_DECISION → Use card or roll
- TURN_OVER → End turn
- GAME_OVER → Game ends

Buttons are enabled/disabled based on phase.

4.2 Callback Architecture

GameEngine uses:

- Consumer<String> log → sends messages to UI
- Runnable onStateChange → refreshes UI

4.3 Jail Mechanic

1. If player has jail card → choose action
2. Use card → exit jail
3. Roll doubles → exit and move
4. Fail → turn ends

4.4 Chance Card System

Possible outcomes:

- Receive \$100
- Receive \$150
- Pay \$50
- Pay repairs
- Advance to GO
- Move forward 3
- Move back 3
- Teleport to property
- Go to jail
- Get jail-free card

Movement uses setPosition() to avoid unintended GO bonuses.

4.5 Board Composition

40-tile board with Vietnamese locations grouped by color sets:

- Brown: Lào Cai, Điện Biên
- Light Blue: Hà Giang, Cao Bằng, Lạng Sơn
- Pink: Hồ Hoàn Kiếm, Phố Cổ Hà Nội
- Orange: Hải Phòng, Nam Định, Ninh Bình
- Red: Thanh Hóa, Nghệ An, Huế, Đà Nẵng
- Yellow: Hội An, Quy Nhơn, Nha Trang
- Green: Đà Lạt, Buôn Ma Thuột, Gia Lai, Kon Tum
- Light Green: Bình Dương, Đồng Nai, TP.HCM, Vũng Tàu
- Dark Blue: Cần Thơ, Phú Quốc, Hà Nội

5. GUI Implementation

Built using Java Swing.

Main layout:

- Board (center)
- Sidebar (player info, dice, buttons, log)

5.1 BoardPanel

Custom-rendered board using Java2D.

- Tile geometry calculated dynamically
- Supports rotated text
- Draws ownership + player tokens
- Optional background image

5.2 DicePanel

- Displays dice images or fallback drawing
- Includes animation using Swing Timer

5.3 PlayerInfoPanel

- Displays player cards
- Shows balance, properties, status
- Highlights active player

5.4 SetupDialog

- Collects player names (2–4 players)
- Dynamic input fields
- Provides defaults

6. Key Design Decisions & Limitations

- No Scanner → fully event-driven
- Phase enum → prevents invalid actions
- Pending purchase flag → avoids blocking UI
- Logging via Consumer → testable design
- Houses removed → simplified gameplay
- Bankruptcy handled in pay() → consistent logic

IV. COMMIT HISTORY

```
* ea1c77a (HEAD -> main, origin/main, origin/HEAD) Final cleanup: Refactored Board class and updated documentation
* 6cd881b docs: complete README with project overview and run instructions
* d2dfc5f docs: complete README with project overview and run instructions
* 3e759fb Final integration: Resolved all merge conflicts and fixed syntax errors in Tile classes
|\
| * 1155dc0 (logic-origin/main) first commit
* f9fd1b6 test gui
```

```
nguyenvoduchuy@uc-Huy-Macbook-Air Test-Monopoly-GUI % git log --pretty=format:"%h - %an, %ar : %s"
ea1c77a - Nguyen Vo Duc Huy, 51 seconds ago : Final cleanup: Refactored Board class and updated documentation
6cd881b - Nguyen Vo Duc Huy, 36 minutes ago : docs: complete README with project overview and run instructions
d2dfc5f - Nguyen Vo Duc Huy, 40 minutes ago : docs: complete README with project overview and run instructions
3e759fb - Nguyen Vo Duc Huy, 41 minutes ago : Final integration: Resolved all merge conflicts and fixed syntax errors in Tile classes
f9fd1b6 - Reeka, 2 weeks ago : test gui
1155dc0 - Nguyen Huy, 2 weeks ago : first commit
```

V. Design Pattern

4.1. Strategy Pattern

- **Application:** Used for the tile system (Property, Chance, Tax, etc.)
- **Mechanism:** The Board class holds a Tile[] array and calls onLand() polymorphically.
- **Benefit:** Adheres to the **Open/Closed Principle**, allowing new tile types to be added without modifying core board logic.

4.2. Singleton Pattern

- **Application:** Managed the Board instance.
- **Mechanism:** Restricts the Board to a single instance to ensure data synchronization across the Logic and GUI components.
- **Benefit:** Maintains global data consistency for player positions and property ownership.

4.3. Encapsulation

- **Implementation:** Critical data (money, position, tile list) is set to private.
- **Benefit:** Protects data integrity by forcing access through getters and setters, allowing for built-in logic like bankruptcy checks.